For any suggestions or feedback regarding these notes,
please contact [Pragy Agarwal]

About the instructor

Pragy Agarwal.
I've been programming for 18+ years now, professionally 7+ years.
Have worked as a frontend dev, backend dev, ML engineer, platform engineer (big data pipelines).
https://www.linkedin.com/in/agarwalpragy/

HLD is a Problem Solving Module

There will be no coding, no deployments during this module.
We will be considering scenarios & case studies – and we will dissect those things.. we will design the solution, and we will solve any problems that arise during the design.

How will we practice the various concepts.

1. Case studies
2. Project Module will happen after HLD (for intermediate + advanced)

If I'm a fresher / I'm applying for SDE-1 (entry-level) positions, then is the HLD module important for me?

Important, but not from an interview perspective for freshers.
You just watch the module like a movie.

No company takes HLD interviews for freshers.
If you have backlogs in the DSA/LLD module ⇒ please first complete those module before you do any of the HLD classes.
90% of your interviews will be DSA, remaining 10% will be LLD+Projects

If I'm applying for senior roles (SDE2+)?

Yes, you need HLD, so please focus on the module.
Ensure that you don't have any backlogs of the DSA / LLD module.

Some companies (like Google) don't take HLD interviews unless you're applying for SDE3+

Ground Rules

1. Before every lecture, please go through the notes for that lecture
2. Any questions should be asked publicly, and put them in the questions tab.

3. Any answers should be sent privately to me
4. Class Timings: 7 am – 9.30 am
5. Doubt Session: 9.30 am - 10.00 am depending on your questions

What & Why of HLD

Low Level Design - Architecture of code

- How to structure the database schema
- The Entities & their Relationships
- The code design patterns
- How are you laying the various components inside your code

LLD helps us keep the code maintainable as the code complexity and team size scales up.

High Level Design - Architecture of the servers / machines that run the code

Interview Question - Staff Engineer @ Google

- 10+ Years of experience
- Salary of Staff Engineer in India (Bangalore, Hyderabad, ...) is 3 Cr+

Given a file containing strings, sort the strings in dictionary (lexicographic) order.

input

`cat, dog, apple, laptop, class, high, level, design`

output

`apple, cat, class, design, dog, high, laptop, level`

Solution

Very very easy!

```
with open('data.txt', 'r') as f:
    data = f.readlines()

print(sorted(data))
```

Catch!

There is **50 Petabytes** of data

1 bit: fundamental unit of information
1 nibble: 4 bits
1 byte: 8 bits
1 Kilobyte: 1,000 bytes

1 Kibibyte (1 KiB) = 1,024 bytes
(Kilo = 1,000 Kibi = 2^{10} = 1,024)

1 Megabyte: 1 million bytes
1 Gigabyte: 1 billion bytes
1 Terabyte: 1 trillion bytes
1 Petabyte: 1 quadrillion bytes

50 Petabytes: 50,000,000 gigabytes!

Q: Will this amount of data fit in your RAM?

No!

Q: Will it fit in your HDD?

No!

This data must be stored in a distributed manner.
Google has more than 10 million servers across the globe.

Collect data => Sort => Store the data back

Q: What can go wrong?

- Network Issues
  - slow
  - down
- Server issues
  - slow
  - down
  - malicious
- Heterogeneity issues
  - different hardware models
  - different operating systems installed
- Human error
  - software is buggy
- Data error
  - hard disk is corrupt

Despite all these challenges you must complete the task, both efficiently, and correctly!

Simple problems become challenging at large scale!
High Level Design is the study of problems that arise at scale, and the solutions for those problems

As we go from 100 users to 1 billion users (planet scale / web scale) what challenges arise, and how we handle them!

Scale

1. Amount of data
2. Number of requests per second

Case Study - del.icio.us A simple bookmarking service

del.icio.us was launched in 2003, created by Joshua

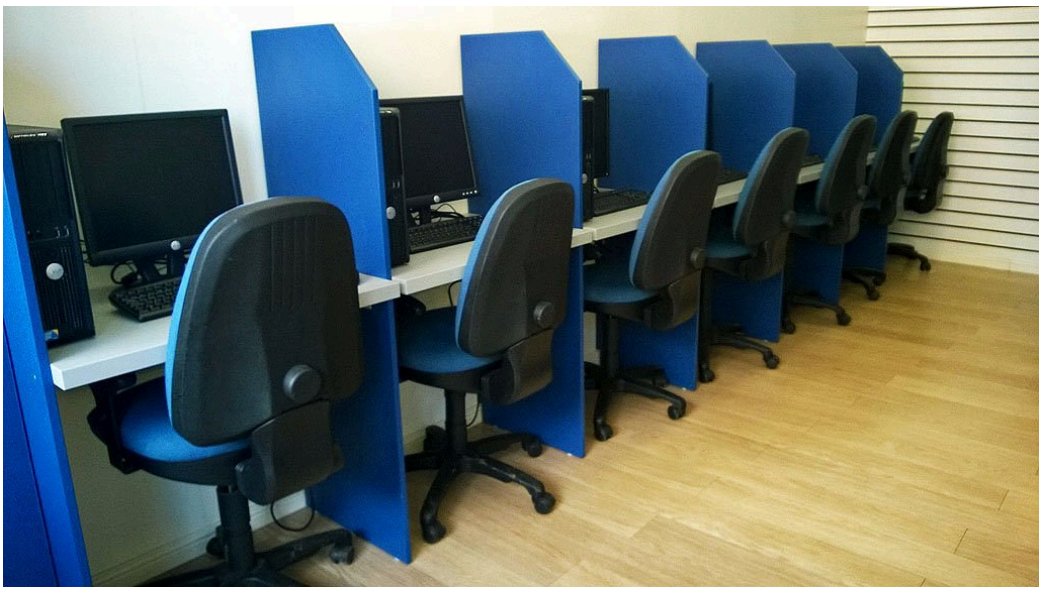
Youtube: 2004
Amazon AWS: 2006
Google Chrome: 2008
Nodejs: 2010

Back when del.icio.us was launched, there was no cloud! People used to buy their own servers!
Browsers were really poor (Internet Explorer)

Motivation

https://en.wikipedia.org/wiki/Delicious_(website)

Most people did not have internet at their home.
Any bookmarks that you took on one system are not available on the other systems - because bookmarks are local to the browser



Let's create a bookmarking service - people can store their personal bookmarks on my website.
My users will be able to access their bookmarks from any system, as long as they have an internet connection.

Minimal Viable Product (MVP)

Product: solves a problem
Viable: usable - demonstrates the solution, should prove the concept
Minimal: minimal set of features needed to be viable *(no fancy features)*

Proof of concept / Prototype

1. **User should be able to identify themselves:** registration + login
   note: logout can be implemented later

2. **User should be able to add bookmarks:** URL

3. **User should be able to view their bookmarks**

deletion / updating / ... can be implemented later. Not needed for MVP.

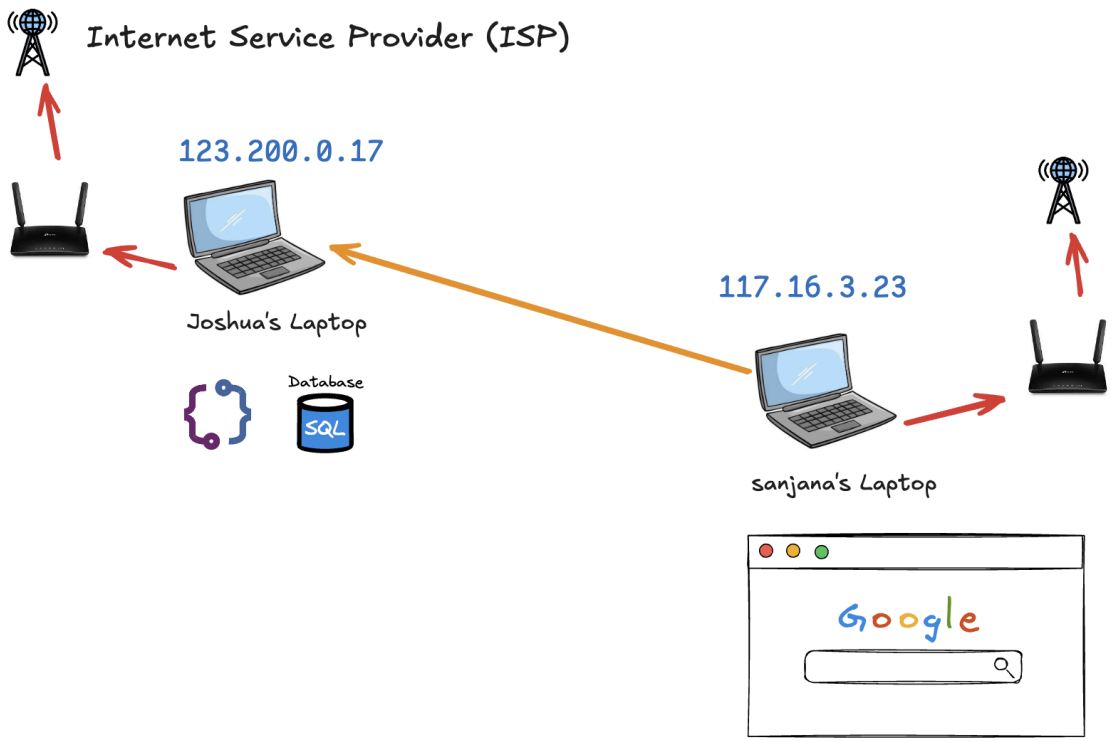
Given this set of features, Joshua can use his favorite tech stack (Python+Django / Java+Spring / PHP+Laravel) and build “webserver” to provide these features.

This webserver will run on his machine “locally”

http://127.0.0.1:5432/
(localhost)

Only Joshua can access this website for now, because it only accessible from his laptop. He wants the rest of the world to be able to access his website.

How Internet Works



Internet Service Provider: your phone company that provides you internet
Airtel / Jio / Vi
ACT / Hayai / Starlink / Comcast

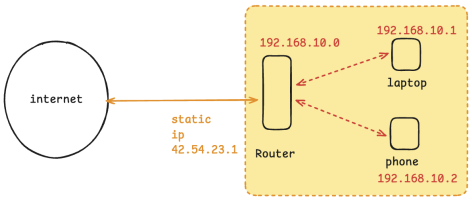
IP address: (kinda) unique address assigned to each machine that is connected to a computer network

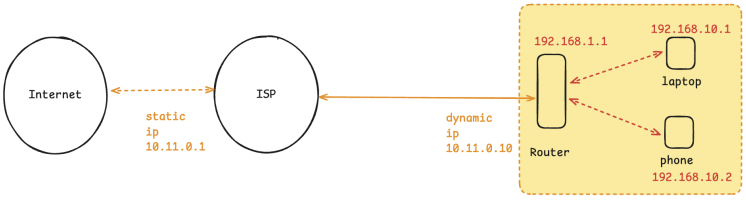
IPv4: `[0-255 : 0-255 : 0-255 : 0-255]` (4 bytes, 32 bits)
How many possible ipv4 addresses? $256^4 = 2^{32}$ ~= 4 billion
Population of Earth, Milkyway: ~8 billion (5 billion have access to internet)
Number of devices on the internet: 100 billion – 1 trillion

IPv6: 16 bytes (128 bits)

How many possible ipv6 addresses? 2^{128} ~ 340 billion billion billion billion (sufficient even if we colonise the entire MilkyWay galaxy!)

- When you connect to the internet via your ISP, the ISP provides you a “Dynamic IP address”
    - you can “purchase” a static ip address if you want, but it is very costly (1 lakh per month)
- Dynamic IP: your true (public) IP address is different, the public ip address is basically of a router that is sitting in your ISP’s building
    - when a request comes for your machine, the request goes to the ISP’s router, and the ISP forwards that request to your router
    - ISP creates a “virtual private network” / “subnet” for all its users in the same location
- How does the router know which machine to forward a packet to, given that all the machines share the same public IP address?
    - packet will contain other information (port number, MAC Address) that will be used to identify the exact machine within the private network that the router has created.





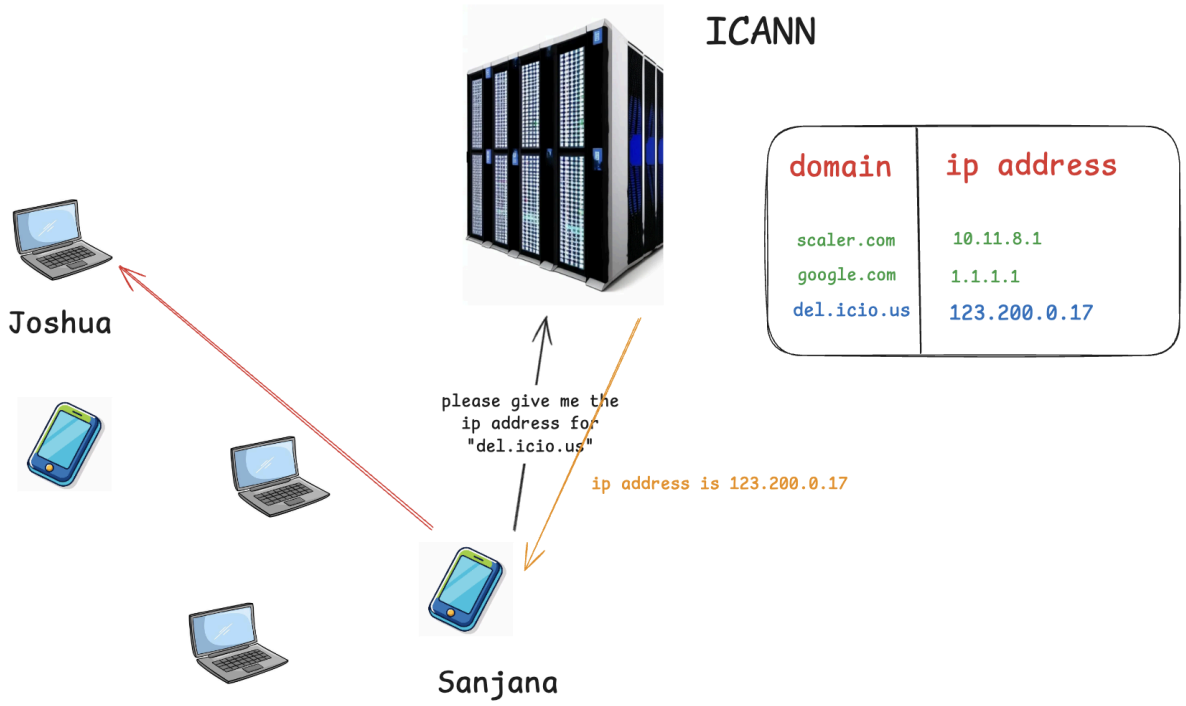
Domain Names

Because IP addresses like 142.250.183.4 are difficult to remember, we need something more “humanized”.
We can purchase “domain names” like google.com
These domain names will be associated with specific addresses.
To purchase a domain name, you’ve to go to a domain registrar (like namecheap.com, godaddy.com) and purchase it from them.

Q: How does the browser know the IP address associated with a website like google.com?

When Sanjana types the website URL in her browser

1. Joshua must purchase a domain
    - Domain Registrar: intermediary that facilitates domain purchase & manipulation
      Godaddy / Namecheap / Cloudflare /
    - Domain registrar will relay that Joshua has purchased a domain to **ICANN**
    - ICANN will store the IP address of Joshua's laptop against the domain name that joshua has purchased



Issues with current design

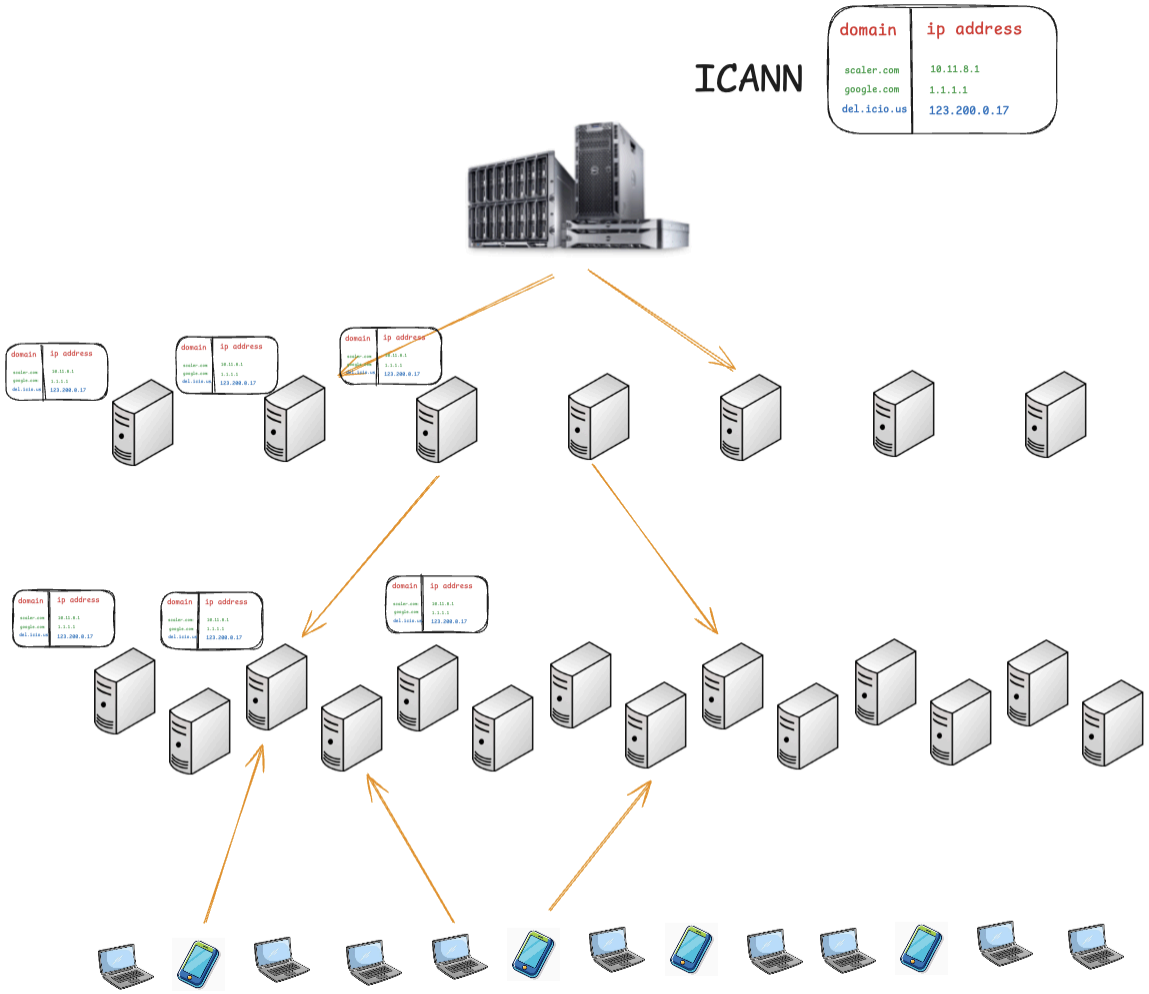
5 Billion of users on the internet
100 billion - 1 trillion devices on the internet

- ICANN servers will become a ***bottleneck***
- ICANN servers will become a ***single point of failure***

```
ICann => I Can't!
```

Domain Name Service (DNS)

DNS maintains ~~an IP address~~ a list of IP addresses for each domain name



Q: Who maintains the DNS?

ICANN is the central authority.
Actual DNS servers are maintained by other entities.

- Anyone can create a DNS server
- Software companies: Google / Amazon / Meta
- Governments
- Militaries
- Research Organizations
- Educational Institutes
- Internet Service Providers (ISP): Airtel / BSNL

You can connect to any DNS server!

Q: What if I do not configure any DNS server in my laptop?

Your ISP also has a DNS server - this is configured in your router by default

Additional Resources (optional)

- Explanation: https://www.cloudflare.com/learning/dns/what-is-dns/
- Google DNS: https://developers.google.com/speed/public-dns
- Deep Dive
  - https://network-insight.net/2015/04/21/dns-structure/
  - https://datatracker.ietf.org/meeting/108/materials/slides-108-tdd-sessb-dns-deep-dive-part-1-pdf-00
- Configuration: https://www.namecheap.com/support/knowledgebase/article.aspx/319/2237/how-can-i-set-up-an-a-address-record-for-my-domain/
- CName: https://support.dnsimple.com/articles/differences-between-a-cname-alias-url/
- Domain extensions (.com .us .in .org)
  - https://en.wikipedia.org/wiki/List_of_Internet_top-level_domains
  - https://www.godaddy.com/resources/skills/most-common-domain-extensions

Del.icio.us - Scaling Challenges

Initially there were 100 users. But after some time, this website went viral.

Joshua was getting millions of new bookmarks each day!

Joshua was initially running this website from his personal laptop

System Configuration (back in 2003)

I got my first PC in 2007

- **Cost:** 35,000 INR
- **RAM:** 128 MB
- **HDD:** 40 GB
- **CPU:** Intel Pentium Core 2 Duo (2 cores, 2.3 Ghz)
- **N/W:** 8Kbps dialup (bsnl)

In 2003, computers were even worse!

Personal Computer vs Server

- When Joshua goes to sleep, he turns his laptop off - website goes down!
- When Joshua downloads a movie / plays a game - website becomes extremely slow!

Joshua must stop using his laptop for his personal use!

Personal computer is for personal use
Server: Dedicated computer whose sole purpose is to provide a (web) service over the internet

Database Design

```
users (id, username, password)

user_bookmarks (user_id, URL)
```

user_id: bigint (8 bytes)
URL: varchar(1000) (1000 bytes)

example of long URL:
https://www.amazon.in/Complete-Balanced-Sardine-Flavour-Pouches/dp/B0BLVJV7VT?pd_rd_w=FoAPt&content-id=amzn1.sym.c6f4280b-256b-4c0a-b056-12ea96d8ae1c&pf_rd_p=c6f4280b-256b-4c0a-b056-12ea96d8ae1c&pf_rd_r=GYW46GCJR692R5YVH0HD&pd_rd_wg=MIjSV&pd_rd_r=57e882f4-82cf-4800-80ac-f1bee983ef09&pd_rd_i=B0BLVJV7VT&psc=1&ref_=pd_bap_d_grid_rp_0_1_ec_nped_pr_pd_hp_d_atf_rp_1_t

4 byte integer

- **unsigned:** 0 ... 2^{32} `(0 .. 4 billion)`
- **signed:** -2^{31} ... $(2^{31} - 1)$ `(-2 billion .. 2 billion)`

4 byte integer is not large enough to store all the users in the world!
only 4 billion possible values for 8+ billion people!

Q: How large is each row in the user_bookmarks table?
1000 bytes (URL) + 8 bytes (user_id)
`= approximately 1 KB`

Q: How much data are we adding to the DB each day if we receive 1 million new bookmarks each day?

(1 kb / ~~bookmark)~~ * (1 million ~~bookmark~~ / day)
= 1 kb * 1 million / day

$= 10^3$ bytes * 10^6 / day
$= 10^9$ bytes / day

= 1 GB / day

Q: In how many days will we run out of disk space?

40 ~~GB~~ / (1 ~~GB~~ / day)

= 40 days

In 40 days, Joshua's server will run out of disk space!

Q: Is disk space the only resource that needs scaling?

Joshua's website is receiving a ton of requests each day

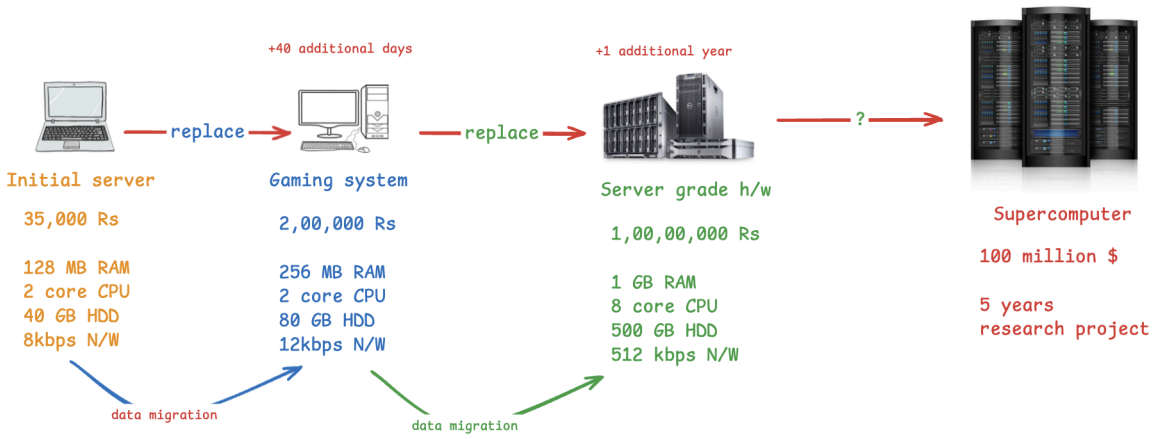
- **CPU** is required: handle the n/w requests, process the db queries
- **Disk** is required: storing the data
- **RAM:** to handle the request contexts
- **N/W:** to handle to requests

Every resource needs to be scaled.

Types of Scaling

1. **Vertical Scaling** (scaling up)

Replace the existing server with a more powerful system



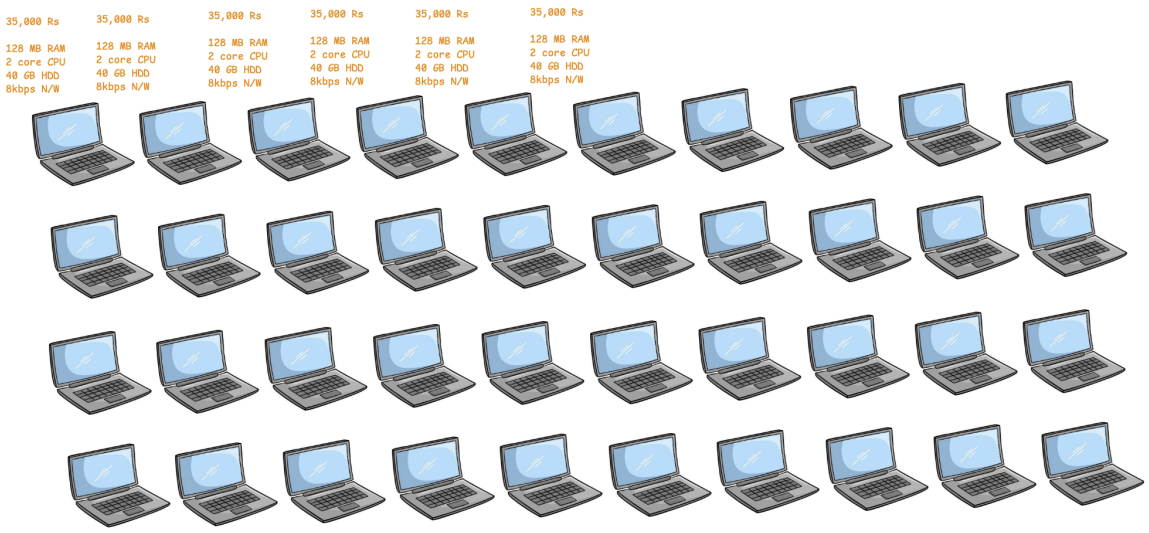
2. **Horizontal Scaling** (scaling out)

Add more machines *along with existing ones*

Alternatively, Joshua could just purchase a lot of cheap laptops

In 1 crore Rs, he can get
`1,00,00,000 Rs / 35,000 Rs` **= 285 laptops!**

In fact he will get more! Because Economy of Scale. He can purchase directly from the manufacturer & get a discount!
Maybe he gets 500 laptops



Which scaling technique should we use?

- Vertical scaling is easy: all you have to do is throw money at it
- But vertical scaling has its limitation: you cannot scale infinitely.
    - After a certain point, you have to scale horizontally
- Vertical scaling is limited by the current technology
    - for example, as of 2025, you cannot get a laptop with 100 exa bytes of storage (this tech will be probably be available in 2035 or 2040)
- Vertical scaling is more costly

- Horizontal scaling is not limited by current technology - you can scale (pretty much) infinitely
- Eventually, you will have to horizontally scale (because vertical scaling will run out)

Horizontal scaling is a huge pain in the ass! It is extremely difficult!

The entirety of the HLD curriculum is about the challenges that arise when scaling horizontally and how to solve them

In reality, you use both. You vertically scale till it's financially feasible and simple. When it becomes too costly, or, when you run out of hardware capability, you scale horizontally.

Certain applications (CPU intensive tasks) mandate both vertical & horizontal scaling

1. **Delicious**: horizontal is all you need. All your systems can be cheap and less powerful
2. **Video Processing:** you have to scale horizontally, but each server itself must have a powerful GPU, CPU, lots of RAM. So, you're also scaling vertically
3. **Large Language Model (ChatGPT):** 7 TB neural network model (you need 7 TB of RAM + 7 TB of GPU memory + 7TB disk). And since you have 500 million users, you also need horizontal scaling

As of Jan 2025, how much can you achieve by vertical scaling?

Typical Server configuration

- **CPU:** `1 - 16 cores`
- **RAM:** `1 GB - 32 GB`
- **HDD:** `500 GB - 8TB`
- **N/W:** `10 Mbps - 1 Gbps`

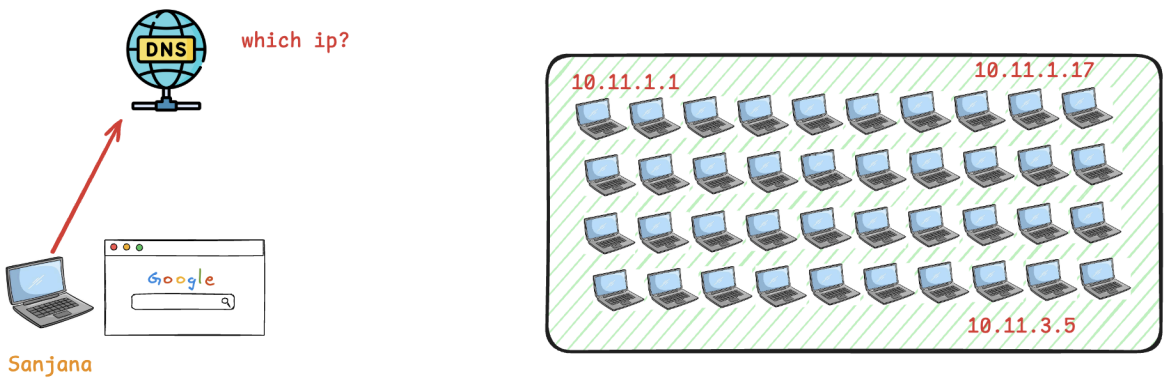
Max Server configuration (Jan 2025)

- **CPU:** `370 cores  (x2 on a single motherboard)`
- **RAM:** `12 TB`
- **HDD:** `2 PB`
- **N/W:** `10 TBps`

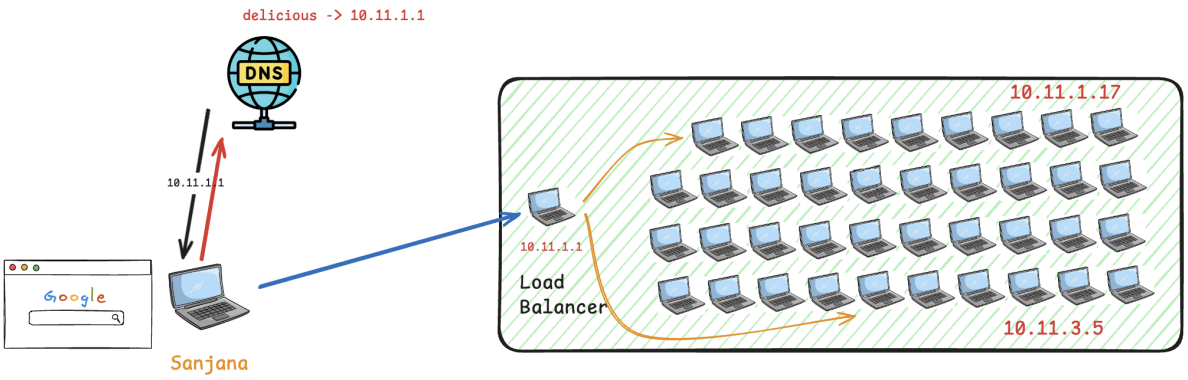
this will cost you you left arm + kidney, your firstborn child, blood of a virgin

Challenges with Horizontal Scaling

Q: Which server's IP should the DNS register?



Load Balancing



Purpose

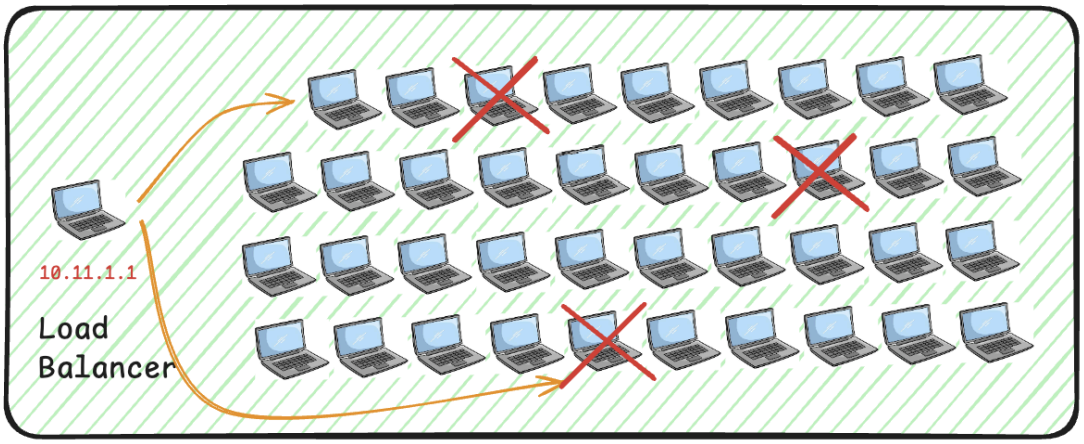
1. provide a unified view of the entire backend system to the end user
   because the end user doesn't are which exact server handles their request

2. **distribute the load (requests/data) equally across the other app/db servers**

The DNS will register the IP address of the Load Balancer (LB)

Q: How does the Load Balancer (LB) keep track of the running servers?

Servers can crash at any time!
The load balancer should not forward a user's request to a server which is down



1. **Heartbeat:** each server periodically pings the LB
   saying "I'm alive!"
   (push based)
   - LB expose an endpoint `/heartbeat`
   - each of the servers will periodically hit this endpoint (every 10 seconds)
   - LB maintains a list of servers that are sending heartbeats
   - If the LB does not receive heartbeats from a server for a few consecutive times, it will assume that the server is dead

2. **Healthcheck:** LB pings each server periodically asks "Are you alive?"
   (pull based)
   - every server exposes an endpoint `/healthcheck`
   - periodically, the LB hits this endpoint for each server
   - If the server fails to respond within time (request timeout), then the LB will assume that the server is dead
   - **How does LB even know which servers exist?**

i. when the server first comes online in the n/w it must ping the LB to register itself
    1. load balancer will expose `/register`
    2. when a new server joins, it will hit this endpoint
ii. you can configure a "range" of IPs in the LB
    `10.11.0.1 ... 10.11.2.20`
    1. LB will try to hit all these ip addresses

Q: Isn't the Load Balancer a bottleneck?

If all the billions of users are sending requests to the LB, won't the LB get overwhelmed?

App server: example - django server to handle bookmarking requests

1. Request go through the *OSI layers* (webserver will reside at the *Application Layer*)
2. Deserialize the request to get the payload
3. Authorization - check the user permissions
4. Fetch the resource from DB
5. Process the data
6. Generate a response
7. Serialize the response
8. Send the response

A typical app server can handle 100 - 1000 requests / sec

Load Balancer: does NOT handle the request - it simply forwards it

1. look at the the IP address in the request
2. decide which server to forward the request to
3. re-route the request to that server

A typical LB can easily handle 100,000+ requests / sec

Therefore, LBs do not easily become bottlenecks.
But at the scale of Google, a single LB will not be enough!
How to solve that?

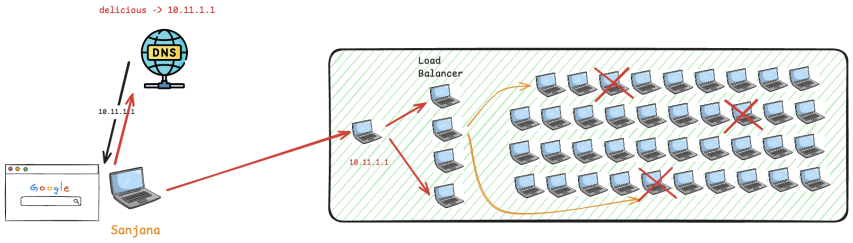
Q: What if the Load Balancer crashes? Isn't the LB a Single Point of Failure (SPoF)?

Yes absolutely! If the LB goes down, the entire website goes down!
How do we fix this?

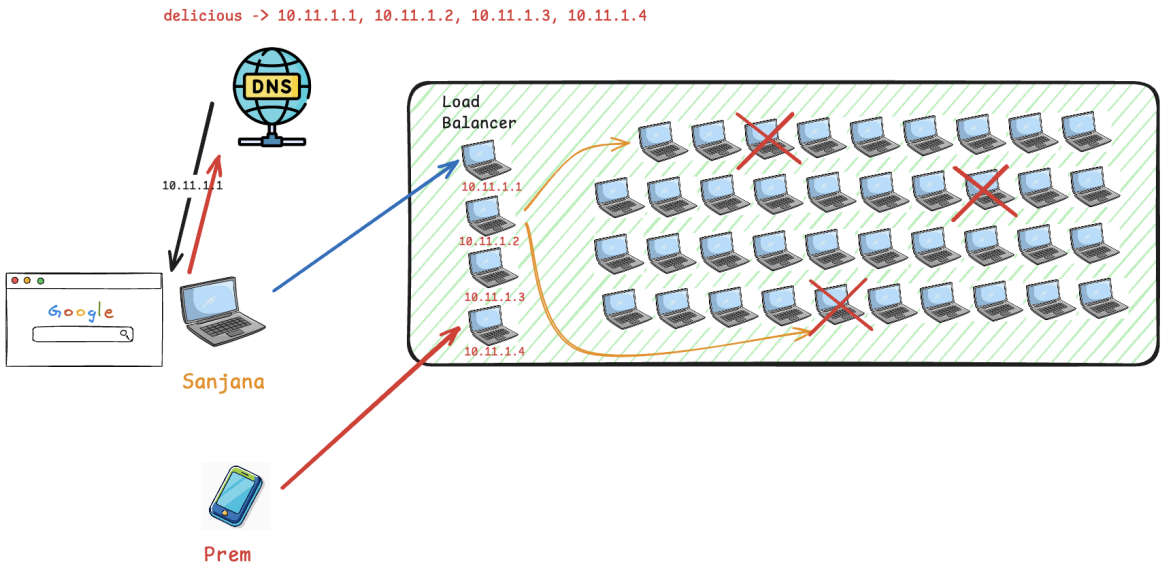
Q: Could we add multiple LBs and have another LB in front of the LBs?

No, that will not work!

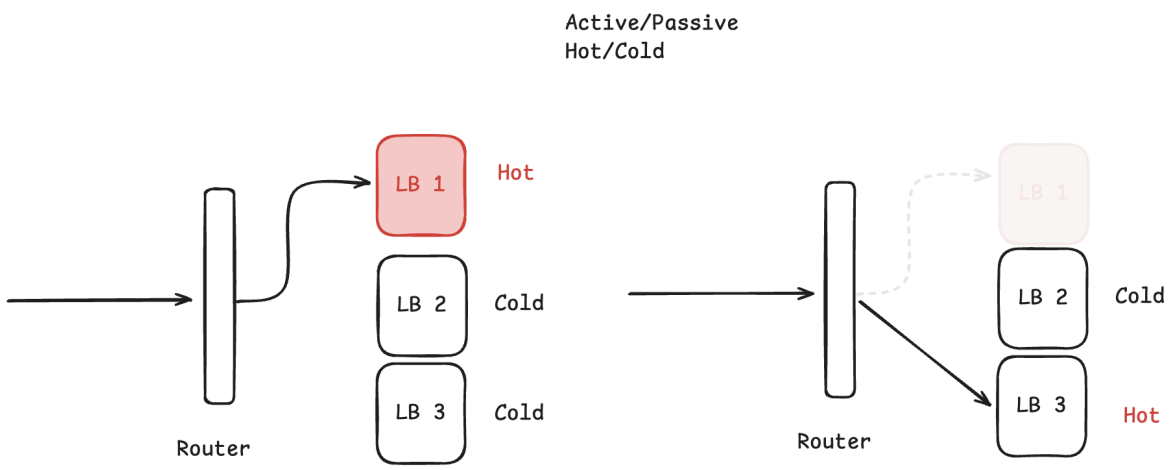
We've just added another layer - the problem now happens at the first LB



Solution: we have multiple LBs, and the DNS acts as a LB in front of the LBs
The DNS can store multiple IP addresses for each domain name.
GeoDNS: it returns the IP address of the LB closest to you
Other DNS servers can simply return the IPs of all the LBs, and then the client can choose themselves (either randomly, or the one which has least latency)



Alternative (not common) — active/passive rollover



Q: Which server should the request be forwarded to?

Routing algorithm decides that!

1. Round Robin
2. **Consistent Hashing**

We will deep dive into the routing algorithms in the next class

HLD Curriculum Overview

➕ Scaler HLD Curriculum & Notes (tentative)

Ensure that you don't have any DSA/LLD backlog (irrespective of your seniority)

SDE 1 - just watch the lectures like a movie
SDE 2 or higher - you need to master HLD

How to succeed in HLD

1. **Be curious**
   This means asking questions during the doubt session, devouring all the reference material that we provide, exploring interesting topics on your own.

2. **Think about the internals**
   For any app/software/solution that you use, think about the internal design, how they might handle the scale, what unique challenges are they facing, and how does all this effect the features they support (and don't support)

3. **Think of the business use-case** of every feature that you consider.
   For example, how does showing a *"blue checkmark for read"* help whatsapp?

4. **Interact with your peers**
   HLD is best learnt via discussions, by bouncing ideas in a group, by talking about what you've seen at work and why, by talking about the unique tech challenges that you and your company are facing.

5. **Solve the assignments before the next class**
   Don't worry, the assignments are MCQs, so they won't take a long time. They will however require critical thinking, which is exactly what we wish to develop in this module.

Additional Resources

Complete list: Scaler HLD Curriculum & Notes (tentative) — *sheet titled "Resources"*

Good resources

- **Reference book:** *"Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems"* by Martin Kleppmann
- **Alex Xu:** ByteByteGo - YouTube
- **Arpit Bhayani:** Arpit Bhayani - YouTube
- **Hussein Nasser:** Hussein Nasser - YouTube
- **Martin Kleppmann:** Martin Kleppmann - YouTube
- **Microservices Reference:** Microservices.io
- **Engineering blogs by companies** like Amazon / Meta / Netflix

Resources to avoid

- Medium articles by random authors (go to engineering blogs of companies instead of engineering blogs of people)
- geeksforgeeks
- random youtube videos by *bhaiyas & didis*

Drawing HLD Diagrams

- I (Pragy) prefer Excalidraw
- A lot of companies use Draw.io

Other Resources

Math Prerequisites (bite-sized lessons)

- **powers:** Expressions with exponents | Algebra basics | Math | Khan Academy
- **logarithms:** Logarithms | Algebra 2 | Math | Khan Academy
- **sequences / series**
    - Sequences | Algebra 1 | Math | Khan Academy
    - Series | Precalculus | Math | Khan Academy
    - **combinatorics (not needed, but good to know):** Counting, permutations, and combinations | Khan Academy

CS Prerequisites (full-length courses)

Do NOT get overwhelmed with these - you are not required to watch these at all.
These are just some extras that you can go through in your free time.

1. **Operating Systems**
    a. UC Berkeley: CS 162: Operating Systems and Systems Programming - Berkeley - YouTube
    b. Scaler: Operating System Course with Certification
2. **Computer Networks**
    a. Northwestern University: CS-340 Intro to Computer Networking - YouTube
    b. Scaler: Computer Networking Course with Certification
3. **Databases**
    a. CMU: CMU Intro to Database Systems (15-445/645 - Fall 2023) - YouTube
    b. Scaler: Free SQL using MySQL Online Course with Certificate

Please note that these are full-length courses. It will take a long time to complete & master these subjects 😁
It is not recommended to go through these while you're doing an interview driven prep.
However, these courses should be in your to-do list in the long run

4. **Distributed Systems:** MIT 6.824 Distributed Systems (Spring 2020) - YouTube

Doubts

Q: Sir, is it mandatory to complete the React and Node.js mock interviews to understand the HLD concepts?

I am looking for entry-level frontend jobs, and I'm planning to complete these two mocks during the HLD module. I have completed all other module mocks and attended all the live classes. In this case, is it still necessary to complete the remaining mocks before attending the HLD sessions?

No, you can do the mocks alongside the HLD module.
Knowing how backend/fullstack code works (express/django/spring) will help you a lot to understand HLD — but for most the HLD curriculum you can understand it even if you're are not very well versed in these concepts.

Beginner: assumption – you've never worked with web projects (webservers). This is why we teach the project module before HLD. HLD is an optional module for beginners.

Intermediate + Advanced: assumption – you've worked a little bit with web projects – so you know the basics of web servers. This is why we cover the project module after HLD module, so we can implement the HLD concepts like containerization, Kafka, Load Testing, Deployments, …

Q:

I have 3 years of experience in IT field in support but don't have prior experience in development so if I apply job outside I will get as junior role or senior ,so I need to focus on hld now or primary focus should be on lld and dsa only and if we skip hld will it affect our psp

You should be aiming for entry position (SDE-1) in tier-1 companies (Amazon, Google)
Pay range ~ 15 LPA to 20 LPA (base)

It could be that you need a ladder — crack 10 LPA/12 LPA job before you can crack a 15 LPA job, but the aim should be a tier-1 company,